AI Engineering and Claude Architect · Part I, Foundations

Generative AI and LLMs, Lesson 6

Why LLMs Fail

A tiny model trained on your policies states "standard customers receive free delivery within 2 days". It's fluent, it's plausible, and it contradicts the text it learned from. Larger models fail more subtly, and Anthropic's research has begun to show what's going on inside.

Module	AI02 of 13
Lesson	6 of 7
Prerequisites	AI02 L4–L5
Exam relevance	High indirect value. Explains TS 4.3 (fabrication), TS 5.1 (position effects), and why self-reported confidence and sentiment are poor triggers (TS 5.2)
Reading time	~40 minutes

Contents

1. Why LLMs Fail

Lesson 6 of 7 · Generative AI and LLM Fundamentals

Why LLMs Fail

Key idea

An LLM generates what is plausible given its training and context, and plausibility isn't truth. This one fact, combined with limited attention and preference-based training, explains hallucination, ignored context, agreeableness and explanations that don't match the real computation. **These aren't bugs that a patch will remove. They're properties to design around**, using grounding, validation, code enforcement and human review.

> *Before: how LLMs are trained (AI02 L4), and how they sample (AI02 L5). Teaches: the main failure modes, their causes and mitigations, and what interpretability research has found. Used by: AI03 (prompting), AI05 (RAG), AI06 L8 (enforcement), AI09, AI10 L3 and AI13.*

The hook: a confident invention

Recall the tiny model from AI02 L4, trained on nine invented policy sentences, including:

> "standard customers receive delivery within 7 days" "gold customers receive free delivery within 2 days"

Real output from generating 200 sentences, using `code/ai02-llm/TinyLm.java`:

```
[NEW]     wallet payments are denied after 14 days of payment
[NEW]     wallet payments are approved within 7 days of payment
[NEW]     gold customers receive refunds are refunded within 2 days
[NEW]     standard customers receive free delivery within 2 days
[NEW]     damaged orders receive delivery within 14 days
[NEW]     orders receive delivery
193 of 200 generated sentences never appeared in training
```

Key idea

Read the fourth line. It's grammatical, it sounds like policy, and it gives standard customers a gold benefit. The model simply stitched together pieces that were each individually likely. Nothing in it represents "true" or "false".

A real LLM is incomparably better, and it's a far harder case to spot. Its inventions are more coherent, and may include convincing details. The underlying relationship is similar: **it produces likely text, and it has no built-in fact check.**

Intuition: a very well-read colleague with no references

Imagine a colleague who's read everything and remembers it through a blurry impression. Ask them something common, and they're nearly always right. Ask them for an obscure case number, and they may give a confident answer that merely sounds right. And when you ask "how did you get that?", they may produce a tidy explanation that doesn't match what actually happened in their head.

Now you'd probably handle them sensibly: trust them with ideas, give them the documents, and check anything that matters. Do the same with an LLM.

The failure map

Failure	What you see	Root cause
Hallucination	Confident, false statements	Plausibility rather than verified lookup
Fabricated fields	Made-up values in JSON	Schema pressure to fill a required field
Context failures	Ignores something you did provide	Limited attention, position effects
Sycophancy	Agrees with a wrong premise	Preference-based training
Unfaithful explanation	Reasoning doesn't match the real process	Explanations are generated text too
Non-determinism	Different results on the same input	Sampling
Stale knowledge	Out-of-date facts	Weights are fixed after training
Overconfidence	Certainty on hard cases	Self-assessment isn't reliably calibrated
Injection	Follows instructions in data	Instructions and data share a sequence (AI09 L2)

Now the most important ones in more depth.

Hallucination

How Anthropic's research explains it. In [Tracing the Thoughts of a Large Language Model], studying Claude 3.5 Haiku, researchers found a surprising structure: a default circuit that causes Claude to **decline** to answer. Recognizing a known entity activates features that inhibit that default.

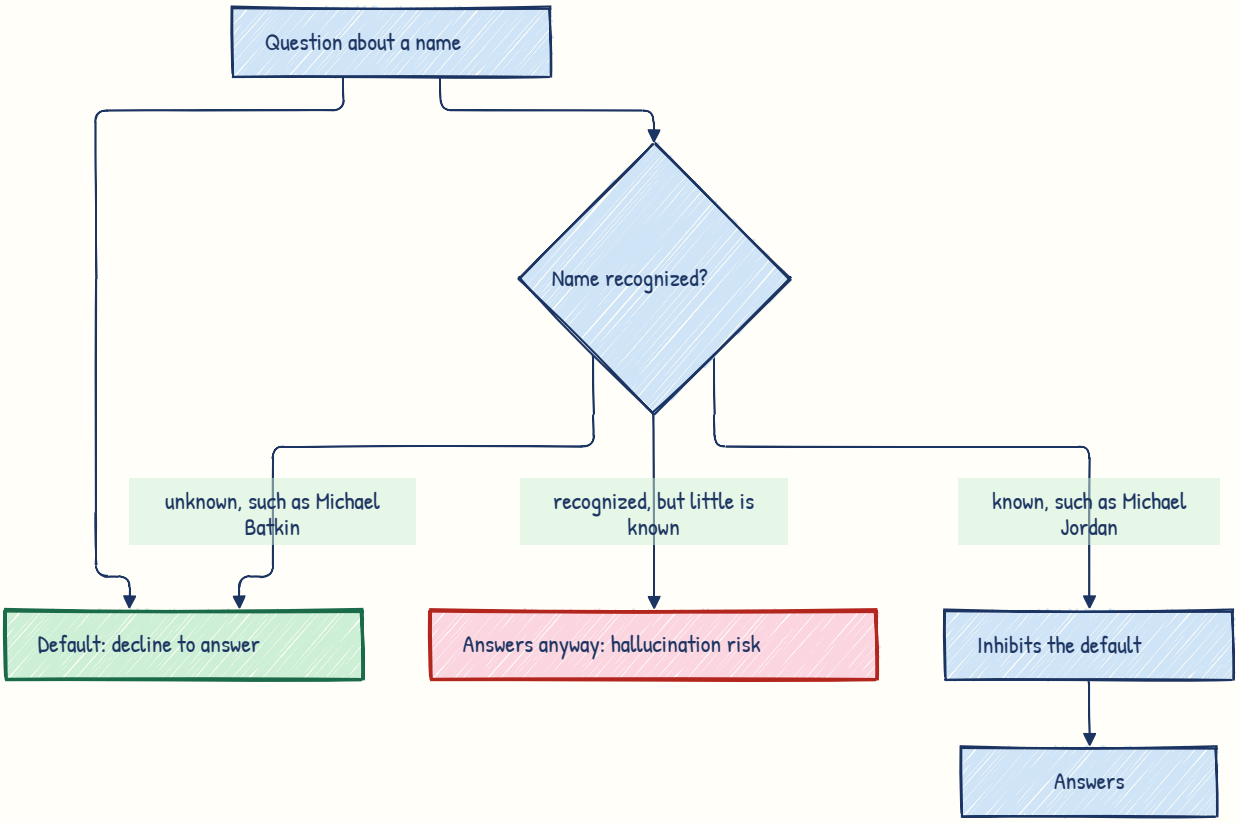


Figure 1. The mechanism reported in Anthropic's research on Claude 3.5 Haiku. A known entity suppresses the default of declining. A hallucination can occur when recognition fires without real knowledge.

The researchers tested this by intervening. When they artificially activated the "known answer" features, Claude consistently claimed that the unknown Michael Batkin plays chess. They also report that this kind of hallucination occurs naturally when a name is recognized, but little else is known.

Key idea

So a hallucination can be a misfire of the system that decides whether it knows something. In other words, it isn't only "making things up". This research applies to a specific model and mechanism, and the authors state its limits below.

Mitigating hallucination

Mitigation	Why it works	Module
Provide sources, and require citations	Grounds the answer in text	AI05 L4
Allow "I don't know" explicitly	Gives the model a valid exit	AI03 L2
Make unknown fields nullable	Removes the pressure to fill them	AI03 L3
Use tools for live facts	Fetches the actual value	AI06 L3
Validate important outputs	Catches what slipped through	AI03 L3

Context failures: "lost in the middle"

Providing the right text doesn't guarantee it's used. Lost in the Middle (Liu and colleagues, 2023), testing multi-document question answering and key-value retrieval, reports that accuracy is often highest when the key information sits at the start or end of the input, and drops noticeably when it sits in the middle. This held even for models designed for long inputs.

The paper studied models of its time, so current models may do better. Measure yours. **The exam still expects you to know the term and the mitigation** (TS 5.1).

Problem	Mitigation
Key facts buried in the middle	Put a summary of key findings at the start
Long, undifferentiated input	Use clear section headers
Many files reviewed at once	Split the work into focused passes. Attention dilution is a term the exam guide uses for this (TS 1.6, TS 4.6)
Bulky tool output	Trim to relevant fields

Sycophancy

Covered in AI02 L4: preference-based training can reward agreement. Anthropic's feature research found something related: a sycophantic-praise feature, which when amplified caused Claude Sonnet to respond to an overconfident user with flattery.

Mitigations: ask neutrally, don't state your preferred answer first, and use independent review instead of asking "are you sure?".

When the model explains itself

This one surprises engineers most. From the same Tracing Thoughts research:

What was tested	What was found
Adding 36 + 59	Parallel paths ran, one approximating and one computing the last digit. Asked how it did this, Claude described the standard carrying algorithm, which didn't match
Square root of 0.64	The reasoning matched the internal computation
Cosine of a large number	It could claim a calculation that left no trace internally
A hint was supplied	It could work backward to reach the hinted answer, which the page calls motivated reasoning

The design rule that follows

A model's explanation is more generated output. It's useful, but it isn't an audit log. Don't treat stated reasoning as proof that a decision was made for that reason. For a regulated decision, record the inputs, the tool calls and the outcome in your own systems.

Self-reported confidence

Anthropic's Language Models (Mostly) Know What They Know (Kadavath and colleagues, 2022) gives a more nuanced picture. It found larger models were well-calibrated on multiple-choice and true/false questions when formatted correctly, and that self-evaluation scores performed well. However,

calibration of "do I know this?" predictions was weak on new tasks.

So the sensible position is: confidence is a signal to calibrate, and not a number to trust. That's precisely the exam's position:

Use of confidence	Exam guide's view
Uncalibrated 1–10 self-score as an escalation trigger	Poor: it's named as an unreliable proxy (TS 5.2)
Field-level scores, **calibrated against a labeled validation set**, used to route review	Good (TS 5.5)

Non-determinism and staleness

Non-determinism: test statistically, make critical logic deterministic in code, and don't rely on exact string matches. **Staleness:** weights are fixed after training, so supply current facts through retrieval or tools, and don't let the model answer "what's today's interest rate?" from memory.

Looking inside: interpretability

Why does this research matter to an architect? Because it changes how we can think about model behavior, and it shows what's still unknown.

Mapping the Mind of a Large Language Model applied a technique called **dictionary learning** to Claude 3.0 Sonnet, grouping recurring patterns of neuron activation into **features**. Each feature maps to a human-interpretable concept. The team extracted millions of them.

Finding	Evidence reported
Features are abstract	A Golden Gate Bridge feature fired on English and on several other languages, and on images of the bridge
Features affect behavior	Amplifying that feature made Claude say it was the bridge
Safety-relevant features exist	Steering a scam-email feature got the model to draft one, despite its harmlessness training

The later Tracing Thoughts work found more:

Finding	Evidence
Planning ahead	When writing rhymed poetry, Claude chose a rhyme word such as "rabbit" before writing the line
Shared concepts across languages	The same features fire for English, French and Chinese
Coherence pressure in a jailbreak	After starting harmful text, grammatical pressure kept it writing until it had finished a coherent sentence

What this research doesn't show

Anthropic is direct about the limits, and you should be too:

→ The method captures only "a fraction of the total computation", even on short prompts.

→ Some mechanisms may be artifacts of the analysis tools.

→ Understanding the circuits takes "a few hours of human effort", even for prompts of tens of words.

→ The features found are "a small subset", and a full set would be "cost-prohibitive" to find.

→ Anthropic says this work hasn't yet shown that safety-relevant features can actually be used to improve safety.

Don't over-apply this

These findings describe particular models, in particular experiments. **They don't mean you can inspect why Claude made a specific decision in your application.** You have no such tool in production, so your design must not assume one.

How Anthropic approaches failure

Failure	Anthropic's published work	What you still have to do
Hallucination	Mechanism research above, and citations support in the API	Ground, cite and validate
Sycophancy	Research, and changes to training	Ask neutrally, and review independently
Harmful output	Constitutional AI, and safety classifiers with a `refusal` stop reason	Handle refusals, and add your own policy
Opacity	Interpretability research	Log and audit in your own systems
Uneven quality	Evaluation guidance	Build evals (AI10 L3)

The production architecture

This is a recommended reference design, not a specific company's system. It answers one question: how do you ship something built on a component that's sometimes confidently wrong?

Baseline: trust the output. The model's answer goes straight to the customer. A fabricated policy becomes a business commitment.

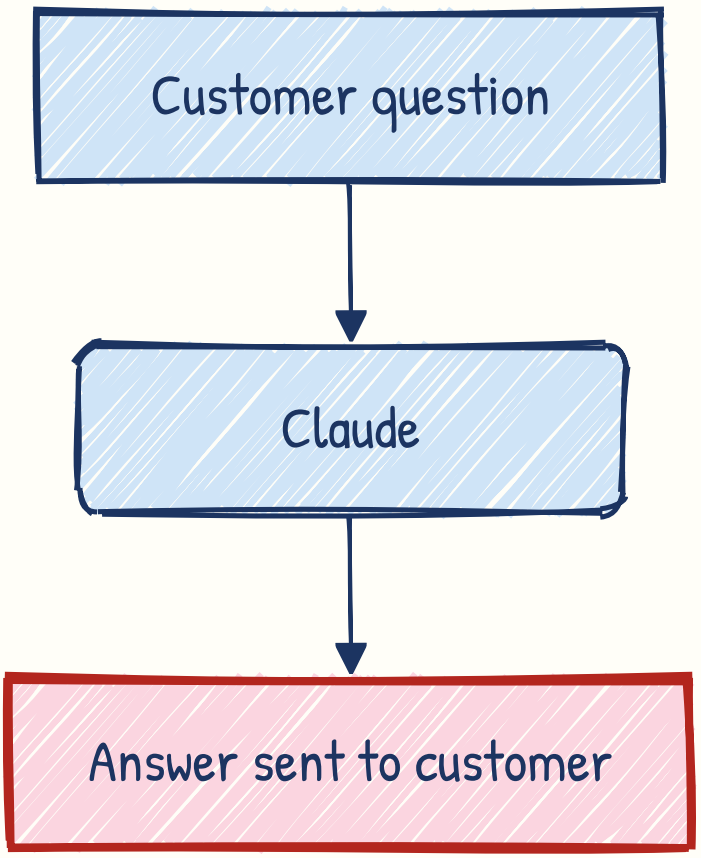


Figure 2. Baseline. Nothing checks the answer, so a plausible invention reaches the customer as if it were policy.

Improved: ground it in documents. Retrieval supplies the actual policy. New problem: **grounding reduces invention without eliminating it.** The model may still misread a document, or ignore one.

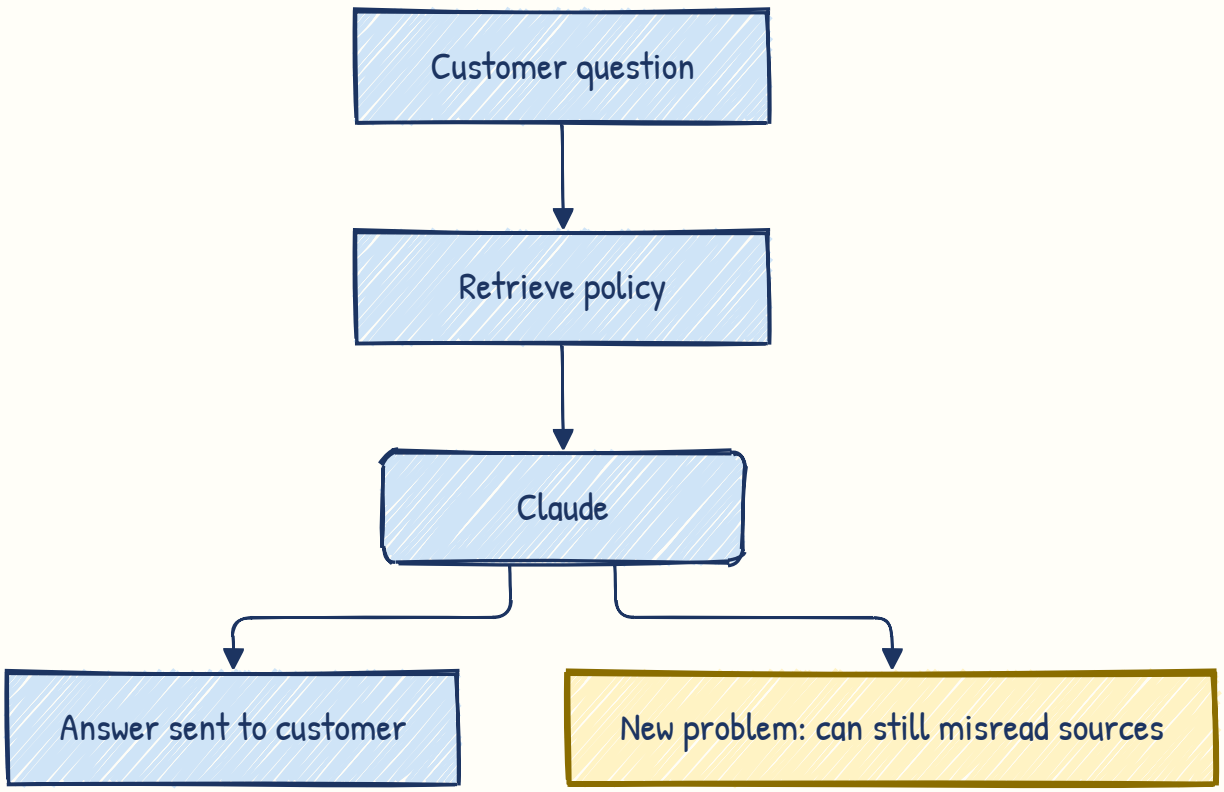


Figure 3. Improved. Answers are now based on real documents, but nothing verifies that the answer follows from them.

Production: defense in depth, by risk. Each layer catches a different failure.

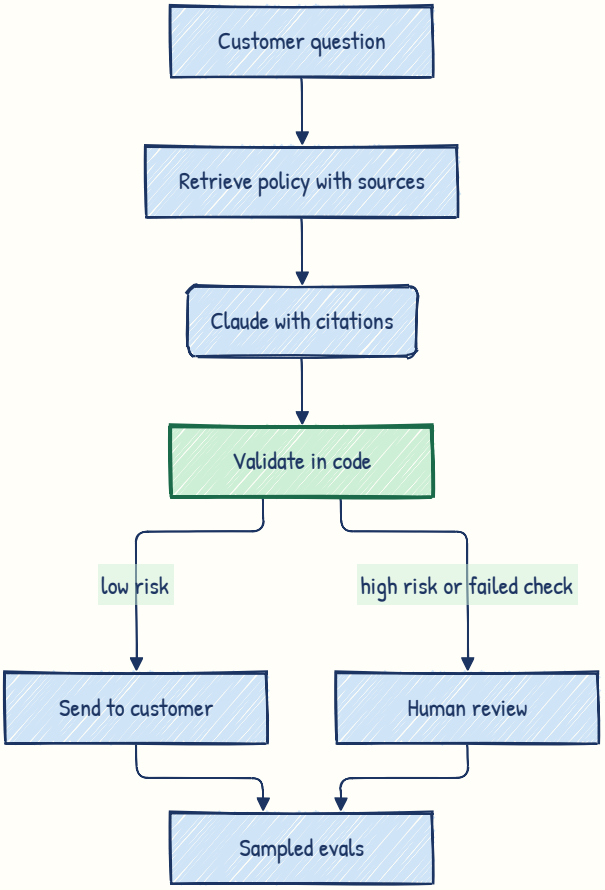


Figure 4. Production. Retrieval reduces invention, validation catches errors in code, and risk decides whether a person checks it. Sampling keeps measuring quality.

Layer	Failure it targets	How it communicates
Retrieval	Invention, staleness	Before the model call (AI05)
Citations	Unverifiable claims	Returned with the answer
Validation in code	Wrong numbers, broken schema, policy limits	Synchronous, deterministic checks
Risk routing	High-cost mistakes	Rules, plus calibrated confidence
Human review	Cases code can't judge	Work queue
Sampled evals	Drift and new failure patterns	Async, stratified sampling (AI10 L3)

Failures

Failure	Containment
A valid citation doesn't support the claim	Show the cited text, and spot-check in evals
A validator has a gap	Sampled human review finds new patterns
Review queue backs up	Prioritize by risk, and shed the low-risk reviews
A model upgrade changes behavior	Re-run evals before release (AI10 L3)

Java

Validation in code is where you, as a Java engineer, add real reliability. This is a simple deterministic check on model output:

```
record RefundDecision(BigDecimal amount, String policyId) {
}

static boolean withinPolicy(RefundDecision decision, BigDecimal orderTotal) {
    return decision.amount().signum() > 0
            && decision.amount().compareTo(orderTotal) <= 0;
}
```

This simple check doesn't care how convincing the model's reasoning sounded. It can't be talked out of its rule. AI03 L3 builds the complete validation and retry loop.

Trade-offs

Decision	Gain	Cost
Retrieval and citations	Fewer inventions, and checkable claims	Pipeline and latency
Validation in code	Guarantees for what's checkable	Only covers what you can express
Human review	Judgment and accountability	Slower and costlier
Requiring "I don't know"	Fewer inventions	May refuse some answerable questions

When a lighter approach is fine: drafting internal text a human will edit anyway, or brainstorming. Match the safeguards to the cost of being wrong.

Common mistakes

1. **Trusting fluent language.** Fluency says nothing about truth.
2. **Treating stated reasoning as an audit trail.** It's generated text.
3. **Using an uncalibrated confidence score to automate decisions.** Calibrate it against labeled data.
4. **Assuming that RAG eliminates hallucination.** It reduces it.
5. **Over-reading interpretability results.** They don't explain your production decisions.
6. **Trying to prompt away a guaranteed rule.** Use code.

What to say in an interview

Asked about hallucination: "An LLM produces plausible text, with no built-in verification. Anthropic's research on Claude 3.5 Haiku found a default circuit for declining, inhibited when a name is recognized, so one kind of hallucination comes from misfired recognition. In practice I'd ground answers, allow abstaining, validate in code and route by risk." A mechanism plus layered mitigation is a strong answer.

Certification lens

How this lesson maps to CCAR-F

Training and internals are out of scope, but the consequences are heavily tested: **fabrication under required fields** (TS 4.3), **position effects** (TS 5.1), **attention dilution** (TS 1.6, TS 4.6), **unreliable confidence and sentiment as triggers** (TS 5.2), and **calibrated confidence for review** (TS 5.5).

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Structured Data Extraction · Medium. Your extraction schema marks `purchase_order_number` as required. Some invoices don't contain one, and the output shows plausible invented numbers. What's the most effective change?

A. Add "never invent values" to the system prompt, and keep the field required.

B. Retry the extraction until a different number appears.

C. Make the field optional or nullable in the schema, so that absence can be represented.

D. Raise the model's reasoning effort for that field.

A **C.** A required field pressures the model to supply something (TS 4.3). Nullable fields remove that pressure. A leaves the pressure in place. B can't find what isn't there (TS 4.4). D doesn't address the schema design.

Q Q2 · Exam style, scenario: Multi-Agent Research System · Medium. A synthesis agent receives 40 findings in one long list. Reports repeatedly omit findings from the middle of the list. Choose the **two** most effective changes.

A. Put a summary of key findings at the start of the input.

B. Organize detailed results with explicit section headers.

C. Add "read everything carefully" to the prompt.

D. Switch to a model with a larger context window.

E. Shuffle the findings' order on each run.

A **A and B.** This addresses position effects directly (TS 5.1). C is a vague instruction. D adds capacity, not attention. E only moves the problem around and makes results less repeatable.

Q Q3 · Case study · Hard. An auditor asks you to show why the assistant denied a loan-related request. Your engineer offers Claude's explanation from the response. What's wrong with that, and what should you have?

A Anthropic's research shows that a model's stated reasoning can differ from its actual computation, so an explanation isn't an audit record. You should have logged the inputs, retrieved documents, tool calls and their results, the rule checks that fired, the model ID and the final outcome. A material decision like this should also rest on deterministic rules and human review.

Revision - Lesson 6

→ LLMs generate what's plausible. Plausibility isn't truth, and the toy model invented 193 of 200 sentences.

→ Anthropic's research on Claude 3.5 Haiku: a default circuit declines, and recognizing a known entity inhibits it. Misfires can cause hallucination.

→ Mitigate hallucination with sources, citations, abstaining, nullable fields, tools and validation.

→ Lost in the Middle: accuracy often drops for information in the middle of long inputs. Put key findings first, and use headers.

→ Sycophancy comes from preference training. Ask neutrally, and review independently.

→ A model's explanation is generated output. It may not match the real computation, and it isn't an audit log.

→ Confidence isn't to be trusted raw. Calibrate it using labeled data before routing on it.

→ Interpretability found millions of features in Claude 3.0 Sonnet, but captures only a fraction of computation.

→ You can't inspect a specific production decision this way.

→ Production: grounding, code validation, risk routing, human review and sampled evals.

Your notes